Post-Training Qwen2.5-0.5B with RLHF

Supervised Fine-Tuning, Reward Modeling, PPO, and Auditable Evaluation

Dheeraj Dhillon

June 2026

Abstract

This report presents an end-to-end reinforcement learning from human feedback (RLHF) study on Qwen2.5-0.5B-Instruct using NVIDIA's HelpSteer3-Preference. The project began with a custom implementation of response-only supervised fine-tuning (SFT), pairwise reward modeling, token-level Proximal Policy Optimization (PPO), KL control, generalized advantage estimation, checkpointing, and policy evaluation. After the custom system exposed practical failure modes, the training loops were migrated to Hugging Face TRL while repository-owned preprocessing, experiment tracking, evaluation, repetition diagnostics, and qualitative curation were retained.

The final pipeline trains on 36,264 filtered preference records with a 4,096-token sequence budget. SFT obtains a validation mean token accuracy of 72.02%. A two-epoch reward model obtains 65.62% pairwise validation accuracy on 1,917 preference pairs. The selected PPO policy is evaluated after 6,400 rollout episodes and 100 optimizer updates. On 2,017 validation prompts, the learned reward model prefers PPO over the original instruction model in 1,027 cases (50.92%), prefers the base model in 981 cases, and assigns 9 ties. The mean PPO-minus-Base reward delta is +0.6497.

That headline is not treated as proof of general model improvement. PPO responses are longer, reach the 1,024-token evaluation cap in 27.42% of cases versus 8.82% for Base, and exceed a 25% repeated 4-gram threshold in 31.88% of cases versus 10.11% for Base. The project therefore contributes a reproducible alignment system and a transparent evaluation case study rather than a claim that PPO universally improves a 0.5B model. All 2,017 prompt and response comparisons are exposed through a public response explorer, with a balanced 100-example subset for faster review.

Contents

1 Introduction

Large language models are trained primarily by next-token prediction, while users ultimately care about properties such as helpfulness, relevance, correctness, clarity, and safety. RLHF addresses this mismatch by learning a reward signal from preference comparisons and then optimizing a language model against that signal [1, 4, 5]. In its canonical form, the procedure has three stages: supervised fine-tuning on preferred demonstrations, training a reward model on chosen/rejected pairs, and policy optimization with a KL constraint to a reference policy.

This project asks a deliberately modest but technically demanding question: can the complete procedure be made measurable and inspectable on a half-billion-parameter instruction model? The small policy makes iteration possible on rented notebook hardware, but it also makes the experiment a stress test of data quality, reward reliability, implementation detail, and evaluation discipline. The base model, Qwen2.5-0.5B-Instruct, is already instruction-tuned and belongs to the Qwen2.5 family [8]. The work is therefore preference post-training rather than instruction tuning from an unaligned base.

The project also connects two settings that are often discussed separately. Earlier work in the same codebase implemented Trust Region Policy Optimization (TRPO), Natural Policy Gradient, and PPO for MuJoCo and Atari. TRPO constrains policy updates through a trust region [2]; PPO approximates the same conservative-update principle with a clipped surrogate objective [3]. In language-model RLHF, a second trust-region-like constraint appears through the KL penalty to a frozen SFT reference. The optimization machinery changes scale, but the central concern remains the same: improve expected reward without allowing destructive policy drift.

1.1 Contributions

The project makes five practical contributions:

1. It implements the full SFT–reward-model–PPO pipeline first with custom training code and then with TRL, preserving the custom path as a reproducible historical baseline.
2. It develops response-safe, long-context preprocessing for HelpSteer3-Preference, including assistant-only loss masks, complete EOS-terminated completions, distinct PAD/EOS tokens, prompt-first truncation, and prompt deduplication for PPO.
3. It incorporates high-impact implementation details from the N+ reproduction of RLHF with PPO [6], including zero dropout, matched behavior probabilities, reward-model initialization from SFT, deliberate scalar-head initialization, reward centering, an RM-initialized critic, fixed-length generation, and strict handling of missing EOS.
4. It evaluates Base, SFT, and PPO on the same 2,017 prompts, stores full responses, supports resumable generation, and derives all pairwise tables from one shared response table.
5. It treats qualitative auditing as part of the result. Repetition, stopping, reward margins, domain effects, and reward-model mismatches are surfaced through deterministic triage, a balanced curated subset, and a public response explorer.

The source repository is available at https://github.com/djdhillxn/rlhf. The response explorer is available at https://djdhillxn.github.io/projects/rlhf-comparison/.

2 RLHF Formulation

2.1 Notation

Let x denote a formatted conversational prompt and $y = (y_1, \ldots, y_T)$ an assistant response. The trainable policy is $\pi_\theta(y \mid x)$, the frozen SFT reference is $\pi_{\text{ref}}(y \mid x)$, and the scalar reward model is

$r_\phi(x, y)$. A response mask $m_t \in \{0, 1\}$ identifies generated assistant tokens so prompt and padding positions do not contribute to policy or value losses.

2.2 Supervised fine-tuning

SFT maximizes the likelihood of the preferred assistant response while masking the prompt:

$$\mathcal{L}_{\mathrm{SFT}}(\theta) = -\sum_{t=1}^{T} m_t \log \pi_\theta(y_t \mid x, y_{<t}). \tag{1}$$

This is ordinary causal language modeling applied only to the completion. It serves two roles: it creates a policy adapted to the preference dataset, and it defines the reference distribution against which PPO drift is measured. Low-rank adaptation (LoRA) [9] updates small trainable matrices inside the attention and feed-forward projections while leaving the main backbone fixed.

2.3 Pairwise reward modeling

Each preference record contains a chosen response y^+ and a rejected response y^- for the same prompt. The reward model is trained with the Bradley–Terry logistic loss:

$$\mathcal{L}_{\mathrm{RM}}(\phi) = -\log \sigma\left(r_\phi(x, y^+) - r_\phi(x, y^-)\right), \tag{2}$$

where σ is the sigmoid function. Pairwise accuracy is

$$\mathrm{Acc}_{\mathrm{RM}} = \frac{1}{N} \sum_{i=1}^{N} \mathbf{1}\left[r_\phi(x_i, y_i^+) > r_\phi(x_i, y_i^-)\right]. \tag{3}$$

The sign of an isolated score is not intrinsically meaningful: adding a constant to every reward leaves Equation 2 unchanged. This project therefore interprets margins and rankings rather than treating zero as a quality boundary. After training, a fixed demonstration-score offset is estimated and subtracted consistently during PPO and evaluation.

2.4 KL-regularized PPO

For a sampled response, the reward model supplies a terminal score while the reference policy supplies a token-level KL-shaped penalty:

$$r_t^{\mathrm{KL}} = -\beta\left[\log \pi_\theta(y_t \mid x, y_{<t}) - \log \pi_{\mathrm{ref}}(y_t \mid x, y_{<t})\right], \tag{4}$$

$$R(x, y) = r_\phi(x, y) + \sum_{t=1}^{T} r_t^{\mathrm{KL}}. \tag{5}$$

The coefficient β controls policy drift. Generalized advantage estimation (GAE) computes

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t), \tag{6}$$

$$\hat{A}_t = \delta_t + \gamma\lambda \hat{A}_{t+1}, \tag{7}$$

with $\gamma = 1$ and $\lambda = 0.95$ in the final run. Advantages are whitened over valid response tokens. For old-policy log probability $\log \pi_{\text{old}}$ and updated-policy log probability $\log \pi_\theta$, define

$$\rho_t(\theta) = \exp\left(\log \pi_\theta(y_t \mid s_t) - \log \pi_{\text{old}}(y_t \mid s_t)\right). \tag{8}$$

PPO minimizes the negative clipped surrogate:

$$\mathcal{L}_{\text{policy}} = -\mathbb{E}_t\left[\min\left(\rho_t \hat{A}_t,\ \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon)\hat{A}_t\right)\right], \tag{9}$$

with $\epsilon = 0.2$. The critic uses a clipped squared-error objective, and the combined optimization loss is

$$\mathcal{L} = \mathcal{L}_{\text{policy}} + c_v \mathcal{L}_{\text{value}} - c_H \mathcal{H}(\pi_\theta). \tag{10}$$

The final configuration uses $c_v = 0.1$. These equations were implemented directly in the custom trainer and later delegated to the experimental TRL PPO trainer. The repository continues to own model preparation, sampling controls, EOS handling, reward centering, checkpoint metadata, and evaluation.

3 Model, Data, and Preprocessing

3.1 Base model and preference data

HelpSteer3-Preference is a human-annotated preference dataset spanning general instruction following, code, STEM, and multilingual tasks [7]. Its breadth is useful for a general alignment study, but it also makes the task harder than short-form summarization: many preferred answers contain code, multi-step explanations, or long conversational context.

Table 1: Final data and model profile.

Item	**Value**
Base policy	Qwen2.5-0.5B-Instruct
Approximate parameter count	0.5B
Raw training preference records	38,459
Raw validation preference records	2,017
Filtered SFT/RM training records	36,264
Filtered SFT/RM validation pairs	1,917
Unique PPO training prompts	24,018
Policy-suite evaluation prompts	2,017
Maximum SFT/RM sequence length	4,096 tokens
Maximum PPO/evaluation prompt length	3,072 tokens
PPO rollout response length	768 tokens
Evaluation generation cap	1,024 tokens

Invalid or tied preferences remove 2,195 training records and 100 validation records from SFT/RM training. PPO uses prompt-only records and removes 14,441 duplicate training prompts. The policy-suite evaluation is separate from this deduplicated PPO prompt cache and covers all 2,017 validation prompts.

3.2 Chat formatting and truncation

All stages use the Qwen chat template and the same tokenizer. The preprocessor constructs three views of each retained preference record:

- **SFT:** prompt plus chosen response, with a completion mask that excludes prompt tokens from Equation 1;
- **Reward model:** chosen and rejected sequences sharing the same prompt prefix;
- **PPO:** deduplicated, left-padded prompt token IDs.

PAD and EOS use distinct token IDs. Every SFT and reward-model completion ends in EOS. When a sequence exceeds the budget, the preprocessor removes the oldest non-system conversational turns first and left-truncates prompt tokens only as a final fallback. It does not cut a response merely to preserve more prompt history.

The project initially used a 1,024-token total limit. A direct length diagnostic showed that this discarded too much preference signal:

Table 2: Historical truncation diagnostic that motivated the 4,096-token training budget.

Limit	Train SFT	Train RM	Val. SFT	Val. RM
1,024	38.47%	40.82%	36.78%	39.49%
2,048	15.48%	16.47%	13.51%	14.87%
3,072	5.28%	5.83%	4.69%	5.32%
4,096	0.83%	1.00%	0.68%	0.89%

The final preparation report is consistent with this diagnostic: 304 of 36,264 SFT training prompts and 363 of 36,264 reward-model training prompts required truncation. The median chosen/SFT sequence length is 768 tokens and the 95th percentile is roughly 3,080 tokens. Long-context handling was therefore not cosmetic; it preserved a substantial part of the dataset.

4 Implementation and Experimental Infrastructure

4.1 From a custom trainer to TRL

The first implementation built the algorithm from low-level components: response masking, shifted token log-probabilities, token-level KL rewards, GAE, normalized advantages, clipped policy and value losses, adaptive KL control, LoRA checkpointing, and evaluation. This path was useful precisely because it failed in visible ways. Short response caps produced truncation; weak KL control or noisy adaptive estimates allowed drift; incorrect checkpoint paths invalidated comparisons; and high reward sometimes coincided with gibberish, empty outputs, or repeated text.

The final system uses Hugging Face TRL for trainer loops and standard serialization [10]. The migration was not a deletion of the project. Responsibilities were divided as follows:

Table 3: Ownership after migration to TRL.

Repository-owned	TRL-owned
HelpSteer3 parsing and filtering	SFT, reward, and PPO trainer loops
Qwen chat formatting and truncation	Mixed-precision/distributed preparation
Completion masks and prompt deduplication	Gradient accumulation and optimizer stepping
Model initialization and reward centering	PPO ratio, clipped objectives, and GAE
Fixed-length generation/EOS patches	Trainer checkpoint serialization
Run manifests and resolved configurations	Core training-loop metrics
Policy-suite evaluation and qualitative audit	PEFT trainer integration

The custom implementation remains frozen at Git commit:

```
6cbf214fcf1b91c7b756e303e533c2c86d2eba89
```

This preserves the learning history and provides a baseline for inspecting how trainer mechanics changed.

4.2 Implementation details adopted from N+

RLHF is unusually sensitive to details that can look incidental in pseudocode. The N+ reproduction [6], which itself reconstructs the summarization setup of Stiennon et al. [4], guided the final configuration:

Table 4: High-impact RLHF/PPO implementation details in the final path.

Detail	Implementation
Disable PPO dropout	PyTorch dropout probabilities and LoRA dropout are zero, preventing stochastic ratio noise between rollout and update.
Match behavior probabilities	Generation temperature is applied consistently when storing and recomputing log probabilities; checkpoint generation heuristics are neutralized.
Preserve completions and EOS	Prompt history is truncated before response tokens; SFT/RM completions end in EOS.
Initialize RM from SFT	The merged SFT model supplies the reward backbone.
Initialize reward head deliberately	The scalar head uses zero bias and standard deviation $1/\sqrt{d_{\text{hidden}}+1}$.
Center rewards	Training uses a centering regularizer, then subtracts a persisted mean demonstration score.
Initialize critic from RM	The PPO value model begins from the trained reward-model weights.
Use a frozen SFT reference	PPO adapter weights are disabled for reference log probabilities.
Fixed-length EOS trick	Rollouts sample the configured length without stopping at EOS, are truncated at EOS afterward, and receive reward -1 if no EOS appears.
Whiten rewards/advantages	Reward whitening is enabled and TRL whitens masked advantages.
Adam numerical setting	Adam epsilon is 10^{-5}.

These choices reduce avoidable instability; they cannot repair a weak reward model, noisy preferences, or insufficient model capacity.

4.3 Reproducibility and Colab execution

Each stage writes a resolved YAML configuration, experiment manifest, summary, trainer metrics, and stage-specific artifacts. Smoke runs precede expensive runs. A “doctor” script validates the

Python interpreter, CUDA visibility, trainer APIs, prepared datasets, tokenizer special tokens, and local/Drive write paths.

The Colab workflow keeps active datasets and checkpoints on local SSD for throughput and synchronizes periodic checkpoints and final artifacts to Google Drive. SFT and reward training support exact Transformers checkpoint resume. The experimental TRL PPO wrapper intentionally does not claim exact resume because the trainer does not expose a reliable optimizer/dataloader resume path in the used API. PPO continuation must instead be recorded as a new segment with explicit parent checkpoints.

Evaluation is resumable at the policy-output level. Each policy writes complete JSONL records as generation proceeds; "partial" denotes interrupt-safe stage output, not truncated response text. A fingerprint guards against reusing cached responses under a changed checkpoint or generation configuration.

5 Final Training Configuration and Results

5.1 Supervised fine-tuning

SFT trains LoRA modules in the attention projections (`q`, `k`, `v`, and `o`) and feed-forward projections (`gate`, `up`, and `down`). The effective batch size is $8 \times 4 = 32$ on one device.

Table 5: Final SFT configuration and outcome.

Setting or metric	Value
Trainer	TRL `SFTTrainer`
Epochs	1
LoRA rank / alpha	16 / 32
LoRA dropout	0
Effective batch size	32
Learning rate	5×10^{-6}
Scheduler	cosine, 3% warmup
Precision	bfloat16, TF32 enabled
Maximum total length	4,096
Train loss	1.0556
Validation loss	1.1127
Validation mean token accuracy	72.02%

"Mean token accuracy" is next-token accuracy over completion tokens. It is not response correctness, instruction-following accuracy, or a direct measure of human preference. The small decline from SFT to Base in the final reward-model comparison later in the report illustrates why token-level validation alone is not a sufficient checkpoint criterion.

5.2 Reward-model training

The reward model loads the merged SFT backbone, attaches a scalar sequence classification head, and trains LoRA rank 32. The first epoch was completed and synchronized; training then resumed from its saved checkpoint to two total epochs. The second-stage configuration uses an effective batch of 64 and centering coefficient 0.01. The persisted raw-score mean on 36,264 preferred demonstrations is -0.1985 (raw-score standard deviation 0.6594), and this offset is subtracted during PPO and evaluation.

Table 6: Final reward-model validation audit.

Metric	Value	Count
Overall pairwise accuracy	65.62%	1,917
Mean chosen-minus-rejected margin	0.3578	1,917
Final evaluation loss	0.6166	1,917
Code accuracy	70.23%	430
General accuracy	62.91%	914
STEM accuracy	59.26%	243
Multilingual accuracy	71.82%	330

Accuracy rises with preference strength: 61.23% for strength 1, 66.33% for strength 2, and 71.10% for strength 3. This is encouraging because clearer human preferences are easier to rank. The low STEM accuracy is the more consequential signal: a general reward model is least reliable in a domain where plausible but incorrect answers can be harmful.

The earlier custom reward model reached 71.62% pairwise validation accuracy, yet its downstream PPO policy did not beat Base. Conversely, the final TRL reward model has lower raw validation accuracy but participates in the best downstream run. This does not imply that lower accuracy is desirable. It shows that one aggregate RM metric cannot summarize initialization, reward scale, EOS handling, critic quality, policy optimization, and out-of-distribution policy outputs.

5.3 PPO alignment

The selected PPO run uses one rollout batch of 64 prompts per optimizer update (2 examples per device with 32 accumulation steps). Each rollout is reused for four PPO epochs. The run was configured for 12,000 episodes, corresponding to 188 planned updates, and was stopped for full evaluation after 6,400 episodes and 100 updates.

Table 7: Final PPO settings and observed training statistics.

Setting or metric	Value
Rollout response length	768 tokens
Rollout batch size	64 prompts
PPO epochs per rollout	4
Policy/value clip ranges	0.2 / 0.2
Value coefficient	0.1
γ / GAE λ	1.0 / 0.95
Sampling temperature / top-p	0.7 / 1.0
KL coefficient β	0.07
Missing-EOS reward	-1.0
Learning rate	3×10^{-6}, linear decay
Reward whitening	enabled
Evaluated episodes / updates	6,400 / 100
Mean response-level objective KL	1.8278
Final response-level objective KL	2.1648
Mean old-to-new approximate KL	0.000640
Mean PPO clip fraction	0.00557
Mean reward-model score in rollouts	-0.5898
Mean EOS count per 64 samples	44.57
Final EOS count per 64 samples	38

The logged `objective/kl` is a response-level sampled sum/aggregate, not a per-token KL target. It should not be compared directly with per-token values such as 0.01 or 0.05 without normalizing by valid response length. The much smaller `policy/approxkl_avg` measures the update from the rollout policy to the post-update policy, whereas `objective/kl` measures departure from the frozen SFT reference. Their different scales are expected.

Negative raw rollout scores are not evidence that PPO receives no learning signal. Reward scores are centered and can be negative for an entire batch; advantages depend on relative returns and value estimates. The final run remained numerically stable: update KL and clip fractions stayed small, the value loss fell from 25.32 on the first update to 5.50 on update 100, and no empty-response collapse appeared. Stability, however, does not rule out reward-model exploitation, which is why evaluation and auditing determine the final interpretation.

6 Experimental Progression

The final configuration was not chosen in one pass. Table 8 summarizes the main iterations without reproducing every debug run.

Table 8: Development path and principal lesson from each phase.

Phase	Experiment	Lesson
Short-context validation	96/128-token generations and small prompt budgets	Fast debugging exposed truncation, blank/EOS collapse, drift, and checkpoint path errors, but could not support credible qualitative evaluation.
Length diagnostic	1,024–4,096 total-token budgets	At 1,024 tokens, roughly 40% of SFT/RM examples truncate; 4,096 reduces this to about 1%.
Custom long-context SFT/RM	4,096-token SFT and two-epoch RM	The custom RM reached 71.62% pairwise accuracy, but judge accuracy alone did not ensure a useful PPO policy.
Custom PPO, 512 evaluation	397 of 400 updates, 512-token cap	Training did not collapse, but PPO won only 44.82% against Base and roughly 30% of outputs hit the cap.
Custom PPO, 1,024 evaluation	Same custom policy, larger generation allowance	Cap hits fell to 11.60% for PPO, while longer outputs exposed repetition, fabricated citations, and technical errors. The comparison was not a clean ablation because batch size also changed.
TRL smoke pipeline	Library-based SFT/RM/PPO with local-SSD and Drive synchronization	Validated stage interfaces, checkpoint loading, merged adapters, and evaluation before the expensive run.
Final TRL run	N+ details, two-epoch RM, 768-token PPO, full 1,024-token evaluation	Produced the first project run in which PPO narrowly beats Base under the learned reward model, while exposing a stronger repetition/stopping cost.

For context, the historical learned-reward comparisons were:

Table 9: Selected policy iterations. These are not controlled ablations: training code, reward models, decoding environments, and batch sizes differ.

Run	PPO vs Base	Mean delta	PPO cap hit	PPO repeat > 25%
Custom, eval cap 512	44.82%	−0.1327	29.70%	–
Custom, eval cap 1,024	38.92%	−0.2137	11.60%	16.11%
Final TRL, eval cap 1,024	50.92%	+0.6497	27.42%	31.88%

Three lessons shaped the final system. First, sequence budgets are part of the learning problem: short caps can hide both useful completions and late degeneration. Second, a reward model must be audited on policy-generated responses, not only held-out preference pairs. Third, small implementation differences – dropout, temperature-adjusted log probabilities, reward scale, EOS treatment, and critic initialization – are algorithmic choices in practice.

7 Final Policy-Suite Evaluation

7.1 Protocol

The evaluator generates one sampled response from each of Base, SFT, and PPO for every one of the 2,017 validation prompts. All policies use the same saved tokenizer, prompt formatting, 3,072-token prompt limit, 1,024-token generation limit, temperature 0.7, top-$p = 1.0$, and no repetition penalty or no-repeat n-gram constraint. The models load sequentially to manage memory. Batch size is 256, prompt order is fixed, and generation can resume from complete per-policy JSONL records.

Every prompt-response pair is scored with the two-epoch reward model. All overall winners, pairwise comparisons, domain tables, plots, and curation files derive from the same response table. This design prevents a pairwise comparison from accidentally using different generations. It does not turn the reward model into a human judge; it measures what the trained proxy prefers.

7.2 Overall results

Table 10: Three-way learned-reward evaluation on 2,017 prompts.

Policy	Wins	Win rate	Mean reward	Mean tok.	Median tok.	Cap hit
Base	718	35.60%	0.0803	398.0	331	8.82%
SFT	508	25.19%	0.0652	436.5	371	13.39%
PPO	775	38.42%	0.7300	577.8	520	27.42%
Tie	16	0.79%	–	–	–	–

Table 11: Pairwise comparisons under the learned reward model. "Right win rate" includes ties in the denominator.

Comparison	Left wins	Right wins	Ties	Right win rate	Mean Δ
Base vs SFT	1,158	837	22	41.50%	−0.0151
Base vs PPO	981	1,027	9	50.92%	+0.6497
SFT vs PPO	840	1,164	13	57.71%	+0.6648

PPO is the most frequent three-way winner and narrowly crosses 50% against Base. The median PPO-minus-Base reward delta is only +0.0254, while the mean is +0.6497. This gap indicates a skewed margin distribution: a subset of large positive deltas raises the mean. Because some extreme positive deltas are repetition failures, the aggregate mean cannot be interpreted without the audit in Section 8.

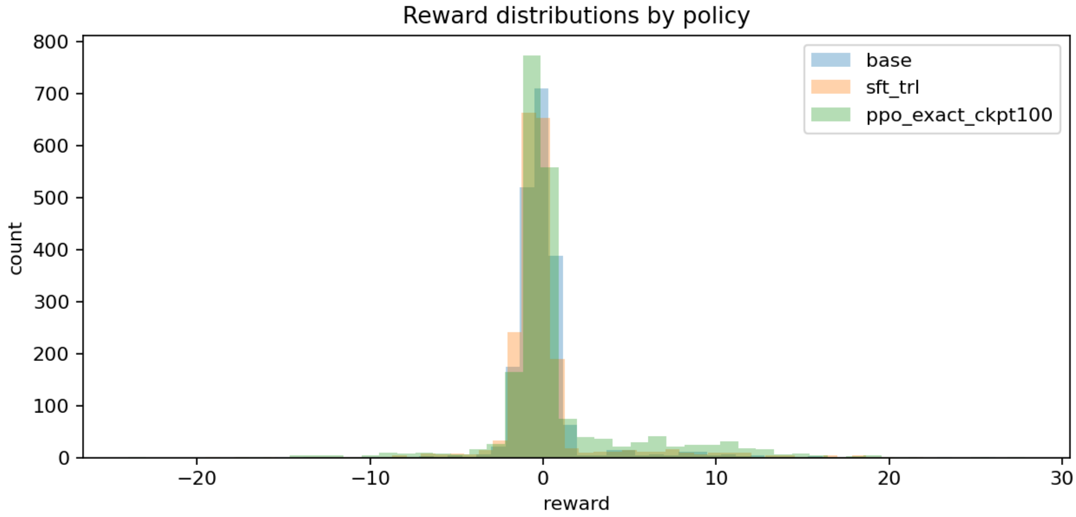

Figure 1: Reward-model score distributions for the three policies. PPO has a heavier positive tail, but distributional separation is limited and extreme scores require inspection.

7.3 Domain behavior

Table 12: PPO versus Base by prompt domain.

Domain	Prompts	PPO wins	Base wins	Ties	PPO win rate
Code	438	188	250	0	42.92%
General	931	529	399	3	56.82%
STEM	245	118	126	1	48.16%
Multilingual	403	192	206	5	47.64%

General prompts account for the overall advantage. PPO loses more often than it wins on code and remains slightly below Base on STEM and multilingual prompts. The domain pattern is consistent with the central limitation of a single generic reward model: it can learn broad stylistic and helpfulness preferences while remaining less reliable on correctness-sensitive tasks.

8 Qualitative Audit and Response Explorer

8.1 Stopping and repetition

The evaluation stores full response text, response length, EOS and cap status, reward margins, and word-level repeated 4-gram diagnostics. For a response with multiset of contiguous word 4-grams $\mathcal{G}$, the repeated fraction is

$$f_{\text{rep4}} = 1 - \frac{|\text{unique}(\mathcal{G})|}{|\mathcal{G}|}. \tag{11}$$

This is a triage feature, not a semantic quality metric. Code and technical prose can repeat legitimate terminology, while a loop can evade the metric by changing punctuation or one word.

Table 13: Stopping and repetition diagnostics.

Policy	**Cap-hit rate**	$f_{\text{rep4}} > 0.25$	$f_{\text{rep4}} > 0.50$
Base	8.82%	204 (10.11%)	61 (3.02%)
SFT	13.39%	405 (20.08%)	171 (8.48%)
PPO	27.42%	643 (31.88%)	319 (15.82%)

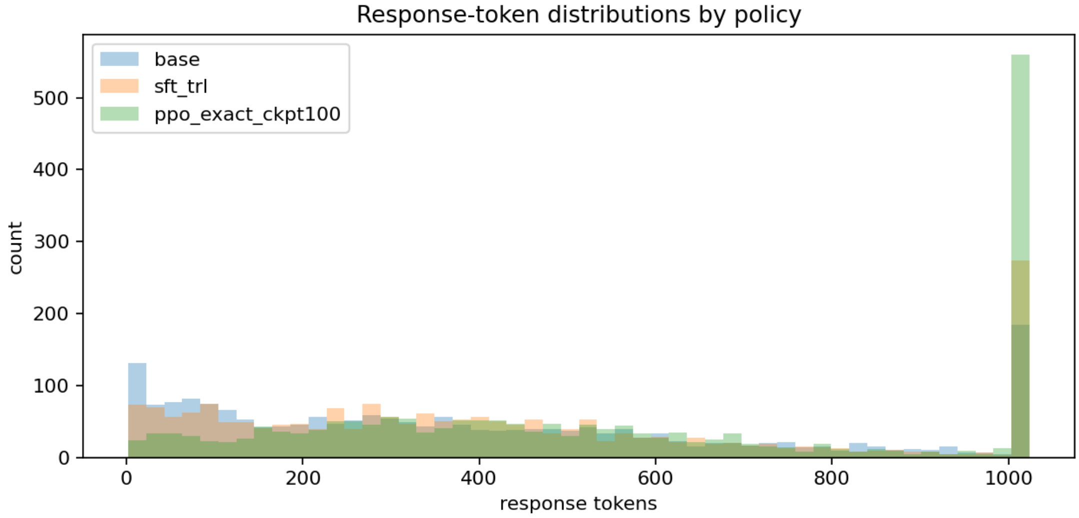

Figure 2: Response-token distributions. The large PPO mass at 1,024 tokens matches its 27.42% cap-hit rate and motivates explicit stopping analysis.

PPO's longer median response can reflect useful additional detail, but it also creates more opportunity to repeat, hallucinate, or continue after the useful answer is complete. The cap spike in Figure 2 shows that the difference is not merely a smooth shift toward slightly longer answers.

8.2 Deterministic triage

Every evaluation row receives one mutually exclusive heuristic label based on reward deltas, EOS/cap behavior, and repetition:

Table 14: Full-set deterministic triage labels. These are navigation aids, not human or LLM-as-judge verdicts.

Label	**Count**	**Meaning**
Likely genuine PPO win	8	PPO beats Base and SFT by a strong reward margin, reaches EOS, avoids the cap, and stays below the repetition threshold.
Modest clean PPO win	354	PPO beats both baselines with low repetition and no cap hit, but with a smaller reward margin.
Needs manual review	924	No deterministic rule makes a strong call.
Repetition risk	228	PPO has elevated repeated 4-gram mass without meeting a more severe rule.
Reward-model false-positive risk	151	The reward model scores PPO highly despite repetition or cap warning signs.
Severe repetition failure	288	PPO reaches the cap with very high repeated 4-gram mass.
Strong PPO regression	64	PPO trails Base by a large reward margin.

Inspection of candidate extremes found both genuine local improvements and clear failures. Positive cases include more supportive responses, better coverage of multi-part instructions, and compact recoveries from capped or meandering baseline answers. Negative cases include repeated loops, prompt restatement, irrelevant continuation, malformed code, fabricated citations, and incorrect scientific procedures. Several high-reward failures are direct evidence of proxy mismatch: the policy discovered features the reward model liked without producing a response a careful reviewer should prefer. This is the practical form of reward-model overoptimization studied more generally by Gao et al. [11].

8.3 Curated 100 and complete public artifact

The public JSON artifact retains all 2,017 full prompts and Base/SFT/PPO responses. Secret-shaped strings are redacted before export. A `curated` flag marks a balanced 100-example first-pass subset:

- 50 positive and 50 negative examples;
- 49 general, 21 multilingual, 19 code, and 11 STEM examples;
- the original reviewed 25 positive/25 negative set is retained;
- the expansion adds clean PPO gains by domain and balances the negative half across 15 reward-model mismatch risks, 15 severe repetition failures, 10 repetition risks, and 10 strong regressions.

The positive half contains all 8 likely-genuine heuristic wins, 41 modest clean wins, and one manually retained compact-recovery edge case whose automatic label remains "needs manual review." Keeping both labels visible is intentional: curated polarity records why the example is informative, while heuristic triage records what the deterministic rule can actually support.

The response explorer loads the full artifact rather than a flattering demo file. Domain filters and the orthogonal Curated filter can be combined, and the page displays full prompt context, complete Base/PPO responses, reward scores, token counts, EOS/cap status, repetition metrics, and heuristic rationale. This makes the evaluation inspectable without requiring a notebook or local model weights.

9 Discussion

9.1 What the final run demonstrates

The strongest supported claim is a systems claim. The project successfully connects long-context data preparation, LoRA SFT, pairwise reward modeling, online PPO, checkpoint-safe execution, full-suite generation, quantitative comparison, and qualitative audit on a real, diverse preference dataset. PPO changes behavior measurably and produces useful local gains. Under the learned reward model, the selected policy narrowly beats Base overall and beats SFT by a larger margin.

The final run also demonstrates why stability is not alignment. Low update KL, small clip fractions, finite losses, and nonzero EOS counts show that optimization is behaving numerically. They do not show that the objective is correct. The repetition and cap metrics reveal a policy that can remain close enough to its reference for PPO diagnostics while still exploiting weaknesses in a scalar judge.

The custom-to-TRL transition was valuable for two different reasons. The custom implementation made algorithmic assumptions visible and produced the failures that motivated better safeguards. TRL then supplied a cleaner, better-tested training substrate for the final run. The project retained its distinct value because data contracts, model initialization, EOS behavior, experiment records, evaluation, and audit remained explicit rather than becoming an opaque library call.

9.2 Limitations

1. **The training judge is also the evaluation judge.** The same reward model supplies PPO rewards and final policy scores. A policy can improve this metric by exploiting judge-specific blind spots. The evaluation is therefore a reward-model-based comparison, not an independent human preference result.
2. **No blinded human study is reported.** Manual review informed curation and interpretation, but it was not a preregistered, multi-rater, label-blind preference study with agreement statistics.
3. **One generation per policy is evaluated.** Sampled decoding at temperature 0.7 has variance. Repeated seeded samples and paired bootstrap confidence intervals would produce stronger uncertainty estimates.
4. **Response length is a confound.** PPO is substantially longer. Longer answers can be more complete, but can also accumulate judge-preferred formatting, unsupported claims, and repetition.
5. **The model and reward model are small.** A 0.5B policy is economical and debuggable, but capacity limits instruction following, factuality, long-context coherence, and value estimation.
6. **The historical comparisons are not clean ablations.** The 512- and 1,024-token custom evaluations differed in batch size and software details, so they document operational learning rather than isolate one causal variable.
7. **The final checkpoint is practical rather than statistically optimal.** Training stopped after 100 updates for evaluation under a deadline. No independent human metric selected it from a checkpoint sweep.
8. **Heuristic auditing is incomplete.** Word-level repetition and a small sensitive-term lexicon can prioritize review but cannot establish factuality, safety, or harmlessness.

These limitations are not footnotes to an otherwise definitive win. They define the boundary of the result.

10 Future Work

The next phase should improve evidence before simply increasing PPO compute.

10.1 Calibrate evaluation with human preference

The highest-value next experiment is blinded review of a stratified sample across domain, reward margin, response length, repetition risk, and curated polarity. Reviewers should compare responses without policy names or reward scores and rate correctness, relevance, completeness, clarity, harmlessness, and unnecessary verbosity. Inter-rater agreement would reveal whether the reward model is wrong or the examples are genuinely ambiguous.

Token-budget evaluations at 512, 768, and 1,024 new tokens should hold checkpoint, precision, batch size, prompt order, seed, decoding, and software fixed. Paired bootstrap intervals and repeated sampled generations would separate policy effects from decoding variance.

10.2 Retrain the reward model on its observed failures

The present audit supplies a hard-negative curriculum: repeated loops, prompt restatements, irrelevant continuations, malformed code, fabricated citations, incorrect STEM procedures, and answers whose useful prefix degrades later. These examples are closer to PPO's optimization distribution than random rejected responses. They should be added to reward training and held-out stress tests.

A stronger evaluator could combine a general preference score with specialized signals for repetition, factuality, code execution, safety, and language consistency. Reward ensembles or multiple seeds could estimate uncertainty; PPO updates could discount examples on which judges disagree. Scaling the reward model before the policy may be safer because a larger policy can exploit a weak judge more efficiently.

10.3 Improve SFT data and stopping behavior

SFT should filter duplicates, broken markup, repeated completions, unsupported citations, and factual errors rather than treating the preferred label as automatic proof of quality. Domain-stratified training and validation would reduce domination by general English prompts. Executable code and verified STEM examples are especially valuable.

Stopping should become an explicit quality dimension. Useful metrics include EOS rate, cap-hit rate, late-response repetition, and reward change between a full response and its prefixes. Dynamic response budgets can reserve long generations for tasks that need them while avoiding 1,024-token continuations for simple extraction or classification prompts.

10.4 Run conservative and comparative policy optimization

A future PPO run should use multi-metric checkpoint selection: reward-model win rate, KL, EOS rate, cap-hit rate, repetition, length, domain correctness, and blinded human preference. A length curriculum from 256 or 512 tokens to 768 and 1,024 may be safer than beginning with the longest horizon. Segment- or process-level rewards could distinguish a useful prefix from a degraded suffix.

PPO should also be compared with preference objectives that avoid online reward-model optimization, especially Direct Preference Optimization (DPO) [12], using the same SFT checkpoint and preference data. RLOO/GRPO-style group-relative baselines are relevant when several responses can be sampled for the same prompt. Such comparisons would help isolate whether the bottleneck is the reward model, the value function, online RL, or the underlying data.

10.5 Scale only after the evaluator improves

Repeating the pipeline at 1.5B or 3B scale would test whether stronger base capabilities turn the same preference signal into more reliable gains. The data and evaluation should remain fixed across scales. Systems work would then include sharding, activation checkpointing, efficient attention, packed sequences, quantized frozen models, and asynchronous rollout generation.

11 Conclusion

This project completes a three-stage RLHF pipeline for a small instruction model and, more importantly, makes the pipeline inspectable. Long-context SFT preserves nearly all preference responses. The reward model learns a usable but imperfect preference signal. PPO remains stable and produces the first run in the project that narrowly exceeds Base under that learned signal. The same evaluation shows that the policy is longer, more frequently capped, and far more repetitive.

The right conclusion is therefore neither “PPO failed” nor “PPO solved alignment.” The project demonstrates that an individual practitioner can build and study the complete RLHF stack, reproduce many subtle implementation details, diagnose reward hacking, and publish the evidence needed to inspect both successes and failures. Its most durable result is the combination of training, evaluation, and honest audit. Future progress depends less on running PPO for a larger number of episodes than on improving the judge, controlling stopping, and validating preferences independently.

A Reproducibility and Artifact Map

Table 15: Primary project artifacts used in this report.

Artifact	Purpose
Project overview	`README.md`. Current project overview and final headline metrics.
Experiment history	`docs/rlhf_experiments.md`. Chronological record from short-context debugging through the final TRL run.
TRL migration	`docs/trl_migration.md`. Trainer ownership, N+ implementation-detail coverage, and Colab/checkpoint contract.
Qualitative audit	`docs/rlhf_qualitative_audit.md`. Interpretation of repetition, stopping, reward mismatch, and candidate pools.
Future work	`docs/rlhf_future_work.md`. Detailed research agenda derived from observed failures.
Prepared data	`rlhf_runs_lightweight_export/.../data/preparation_report.json`. Filtered counts, truncation counts, and sequence-length distributions.
SFT summary	`rlhf_runs_lightweight_export/.../sft/run_summary.json`. Final SFT training summary.
Reward audit	`rlhf_runs_lightweight_export/.../reward_epoch2/reward_audit.json`. Final reward-model accuracy, margins, and domain/language breakdown.
PPO trainer state	`rlhf_runs_lightweight_export/.../checkpoint-100/trainer_state.json`. Selected PPO training history through 100 updates.
Policy-suite summary	`rlhf_runs/checkpoints_ckpt100_full/policy_suite_summary.json`. Authoritative final policy and pairwise evaluation.
Full response artifact	`rlhf_runs/portfolio_full_policy_comparisons_final_trl.json`. All 2,017 full prompts and policy outputs, heuristic labels, and curated flags.
Curated manifest	`rlhf_runs/portfolio_curated_100_manifest_final_trl.json`. Compact manifest of the balanced curated subset.

The active command sequence is:

```
python scripts/rlhf_trl_prepare_data.py \
  --config configs/trl/qwen25_05b_helpsteer3_sft.yaml
python scripts/rlhf_trl_train_sft.py \
  --config configs/trl/qwen25_05b_helpsteer3_sft.yaml
python scripts/rlhf_trl_train_reward_model.py \
  --config configs/trl/qwen25_05b_helpsteer3_reward.yaml
python scripts/rlhf_trl_train_ppo.py \
  --config configs/trl/qwen25_05b_helpsteer3_ppo.yaml
python scripts/rlhf_evaluate_policy_suite.py \
  --config configs/trl/qwen25_05b_helpsteer3_eval_suite.yaml
```

The executed Colab notebook is the source of truth when a directory name or base YAML conflicts with a runtime override. In particular, the PPO directory retains an older `r512` label, while the selected run used 768-token rollouts and KL coefficient 0.07.

B Metric Interpretation

SFT mean token accuracy
Fraction of preferred completion tokens predicted correctly under teacher forcing; not response-level task accuracy.

Reward-model pairwise accuracy
Fraction of held-out pairs for which the chosen response scores above the rejected response.

Policy win rate
Fraction of evaluation prompts for which one policy receives the higher learned reward. It is a proxy-judge result, not a human preference rate.

Objective KL
Sampled response-level departure from the frozen SFT reference. It is not the same statistic as PPO's old-to-new update approximate KL.

Cap hit
The response uses the full evaluation generation budget and does not stop naturally before 1,024 new tokens.

Repeated 4-gram fraction
Fraction of word-level 4-gram positions attributable to repeated 4-grams, used only to prioritize qualitative review.

References

[1] P. F. Christiano, J. Leike, T. Brown, M. Martic, S. Legg, and D. Amodei. Deep reinforcement learning from human preferences. In *Advances in Neural Information Processing Systems*, 2017. https://arxiv.org/abs/1706.03741.

[2] J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz. Trust Region Policy Optimization. In *Proceedings of the 32nd International Conference on Machine Learning*, 2015. https://arxiv.org/abs/1502.05477.

[3] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal Policy Optimization Algorithms. arXiv:1707.06347, 2017. https://arxiv.org/abs/1707.06347.

[4] N. Stiennon, L. Ouyang, J. Wu, D. Ziegler, R. Lowe, C. Voss, A. Radford, D. Amodei, and P. F. Christiano. Learning to summarize with human feedback. In *Advances in Neural Information Processing Systems*, 2020. https://arxiv.org/abs/2009.01325.

[5] L. Ouyang et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, 2022. https://arxiv.org/abs/2203.02155.

[6] S. Huang, M. Noukhovitch, A. Hosseini, K. Rasul, W. Wang, and L. Tunstall. The N+ Implementation Details of RLHF with PPO: A Case Study on TL;DR Summarization. arXiv:2403.17031, 2024. https://arxiv.org/abs/2403.17031.

[7] Z. Wang, J. Zeng, O. Delalleau, H.-C. Shin, F. Soares, A. Bukharin, E. Evans, Y. Dong, and O. Kuchaiev. HelpSteer3-Preference: Open Human-Annotated Preference Data across Diverse Tasks and Languages. arXiv:2505.11475, 2025. https://arxiv.org/abs/2505.11475.

[8] Qwen Team. Qwen2.5 Technical Report. arXiv:2412.15115, 2024. https://arxiv.org/abs/2412.15115.

[9] E. J. Hu et al. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations*, 2022. https://arxiv.org/abs/2106.09685.

[10] L. von Werra et al. TRL: Transformer Reinforcement Learning. Hugging Face, 2020–2026. https://github.com/huggingface/trl; https://huggingface.co/docs/trl/ppo_trainer.

[11] L. Gao, J. Schulman, and J. Hilton. Scaling Laws for Reward Model Overoptimization. In *Proceedings of the 40th International Conference on Machine Learning*, 2023. https://arxiv.org/abs/2210.10760.

[12] R. Rafailov, A. Sharma, E. Mitchell, C. D. Manning, S. Ermon, and C. Finn. Direct Preference Optimization: Your Language Model is Secretly a Reward Model. In *Advances in Neural Information Processing Systems*, 2023. https://arxiv.org/abs/2305.18290.